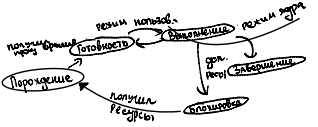


1. 2.1. Лаба UNIX: создание процесса в ОС UNIX и запуск новой программы (сх: именные вызовы);
Примеры программ из лабораторных работ, демонстрирующие эти действия. Процессы - «сироты», «зомби» и «демоны». Системные вызовы fork(), wait(), pipe(), signal() и sigaction();
назначение, примеры из лабораторных работ. Сравнение средств IPC: программных каналов, очередей сообщений и разделяемых сегментов.



Процесс - основная абстракция ОС для декомпозиции работы программы, основная единица декомпозиции.
Процесс - программа в стадии выполнения.
С точки зрения UNIX процесс часть времени выполняется в режиме пользователя (в режиме ядра), тогда он выполняет собственный код, и часть в режиме ядра, тогда он выполняет рентерабельный код ОС.
Рентерабельный код - код повторной входимости (код чистых процедур).

В ОС нет таблицы процессов - есть связанный список, task_struct - дескриптор процесса.
Порядок инициализации процесса: добавление в связанный список и дескриптора (большая структура); struct proc/task_proc в UNIX BSD/LINUX большинство полей - указатели на другие структуры. В Unix часть полей дескриптора процесса инициализируется значениями полей процесса-предка. Чтобы процесс начал выполняться, ему нужно выделить физическую память.
После того, как процесс получил память, он переходит в состояние готовности. Планировщик - специальная часть кода ядра, организующая очередь готовых процессов в соответствии с принятой дисциплиной планирования. Процесс выделяется min необходимое число страниц. В состоянии готовности процесс обладает всеми ресурсами, кроме процессорного времени. Требуется дополнительные ресурсы -> блокировка (I/O, semaphore).
Переход в режим ядра происходит при: 1) системных вызовах (программное прерывание); 2) исключении/ошибках кода системы (бывает несправимыми и исправимыми); (1, 2) - синхронные; 3) аппаратных прерываниях - прерывания от системного таймера, от I/O устройств (происходит после завершения операции I/O), от действий оператора (асинхронные).

fork()
Системный вызов fork() создает копию процесса. Полученный процесс является потомком, а первоначальный - предком (parent-child). Адресное пространство и программа (код) родителя и потомка идентичны. После вызова fork() потомку возвращается PID=0, а родителю PID=ChildID.

Алгоритм работы fork()
Для создания нового процесса системный вызов fork() выполняет следующие действия:
1. Резервирует память под данные и стек
2. Назначает идентификатор PID и структуру проца
3. Инициализирует структуру проца. Некоторые поля копируются из структуры предка.
4. Создаются карты трансляции адресов
5. Выделяется u-область, куда копируется информация из 'u-область' и предка
6. В 'u-область' изменяются ссылки на новые карты адресации и пространство под данные и стек
7. Процесс добавляется в набор процессов, разделяющих адресное пространство с предком
8. Секции данных и кода предка постранично дублируются в пространство потомка
9. Потомок получает ссылки на разделяемые ресурсы
10. Потомок помещается в очередь готовых процессов
11. В точку вызова fork() возвращается PID
Оптимизация fork()
Для эффективного управления памятью был введен механизм copy-on-write. При создании потомка страницы пространства предка не копируются, а получают статус READONLY. Копирование страниц происходит только в момент, когда предок или потомок захотят изменить соответствующую страницу. Данный механизм сокращает количество копированных данных.

Процесс сирота
Если предок завершается раньше потомка, то происходит процесс усыновления, когда предком дочерних процессов становится процесс с идентификатором 1, либо процесс посредник.

Процессы «зомби»
Применение вызова fork() для создания процессов может оказаться очень полезным, но необходимо отслеживать дочерние процессы. Когда дочерний процесс завершается, связь его с родителем сохраняется до тех пор, пока родительский процесс в свою очередь не завершится нормально, или не вызовет wait(). Следовательно, запись о завершении процесса не исчезает из таблицы процессов немедленно. Становясь неактивным, дочерний процесс все еще остается в системе, поскольку его код завершения должен быть сохранен, на случай если родительский процесс в дальнейшем вызовет wait(). Процессы-зомби - это процесс, у которого отобраны все ресурсы, кроме последнего - строки в таблице процессов. Это сделано для того, чтобы процесс-предок, вызвавший системный вызов wait(), не был заблокирован навсегда. Можно увидеть переход процесса в состояние зомби, если дочерний процесс завершается первым, то он будет существовать как зомби, пока процесс-предок или вызовет системный вызов wait(), или родительский процесс завершится.

Демон - это процесс, работающий в фоновом режиме. Такой процесс не имеет управляющего терминала. Пример из лабы в самом конце.

exec()
Системный вызов exec() заменяет адресное пространство исполняемого процесса на адресное пространство новой программы. Новый процесс не создается, потому идентификатор процесса не меняется, но код, данные, куча и стек, естественно, заменяются.

Алгоритм работы 'exec()':
1. Получить доступ к переданному файлу
2. Проверить права доступа активного процесса к файлу
3. Анализ заголовка. Проверка, что файл исполняемый
4. Изменение идентификатора процесса на идентификаторы переданной программы при установке определенных флагов
5. Копирование переменных окружения и аргументов exec() в пространство ядра. Прежнее пространство готово к удалению
6. Выделяется пространство для данных и стека
7. Выделяются карты трансляции адресов
8. Выделяется новое адресное пространство
9. Переменные окружения и аргументы exec() копируются в стек процесса
10. Сброс обработчиков сигналов в действия по умолчанию
11. Инициализация аппаратного контекста. В IP точка входа в программу, в CR3 адрес таблицы страниц

Тут пример 3
wait()
Данный системный вызов блокирует родительский процесс до момента завершения дочернего процесса. При этом процесс-предок получает статус завершения процесса-потомка. pid_t wait(int *stat_loc); Вызов возвращает PID дочернего процесса. Обычно это дочерний процесс, который завершился. Сведения о состоянии позволяют родительскому процессу определить статус завершения дочернего процесса, т.е. значение, возвращенное из функции main потомка или переданное функции exit(). Если stat_loc не равно пустому указателю, информация о состоянии будет записана в то место, на которое указывает этот параметр. Интерпретировать информацию о состоянии процесса можно с помощью макросов:

WIFEXITED(stat_val) - Ненулевой, если дочерний процесс завершен нормально
WEXITSTATUS(stat_val) - Если WIFEXITED ненулевой, возвращает код завершения дочернего процесса
WIFSIGNALED(stat_val) - Ненулевой, если дочерний процесс завершается не переживаемым сигналом
WTERMSIG(stat_val) - Если WIFSIGNALED ненулевой, возвращает номер сигнала
WIFSTOPPED(stat_val) - Ненулевой, если дочерний процесс остановлен
WSTOPSIG(stat_val) - Если WIFSTOPPED ненулевой, возвращает номер сигнала

Тут пример 2
pipe()
Каналы реализуют потоковую передачу данных - симплексная связь. Системный вызов pipe() создает неименованный программный канал. Неименованные программные каналы могут использоваться для обмена сообщениями между процессами родственниками. В отличие от именованных программных каналов неименованные не имеют идентификатора, но имеют дескриптор. Процесс-потомок наследует все дескрипторы открытых файлов процесса-предка, в том числе и неименованных программных каналов.

Тут пример 4

2. Лаба UNIX: создание процесса в ОС UNIX и запуск новой программы (сх: именные вызовы);
Примеры программ из лабораторных работ, демонстрирующие эти действия. Процессы - «сироты», «зомби» и «демоны». Системные вызовы fork(), wait(), pipe(), signal() и sigaction();
назначение, примеры из лабораторных работ. Сравнение средств IPC: программных каналов, очередей сообщений и разделяемых сегментов.

#include <signal.h>
static int flag = 0;
void signal_handler(int sig_num)
{
flag = 1;
printf("Signal %d received. Sending messages is allowed.\n", sig_num);
}
int main(void)
{
pid_t child_pid[2];
char *messages[2] = {"aaaa", "bbbb"};
int fd[2];
pipe(fd);
signal(SIGINT, signal_handler);
printf("Press Ctrl+C to allow sending messages\n");
for (size_t i = 0; i < 2; i++)
{
child_pid[i] = fork()
else if (child_pid[i] == 0)
{
printf("Child");
if (flag)
{
printf("Child sent message: %s", messages[i]);
close(fd[0]);
write(fd[1], messages[i], strlen(messages[i]));
}
else
printf("Child didn't send message\n");
return 0;
}
printf("Parent");
for (size_t i = 0; i < 2; i++)
{
int status;
waitpid(child_pid[i], &status, 0);
if (WIFEXITED(status))
{
printf("Child process has finished with code = %d", WEXITSTATUS(status));
}
else if (WIFSIGNALED(status))
{
printf("Child process has finished from signal with code = %d", WTERMSIG(status));
}
else if (WIFSTOPPED(status))
{
printf("Child process has stopped with code = %d", WSTOPSIG(status));
}
}
printf("Messages from children: \n");
for (size_t i = 0; i < 2; i++)
{
char buf[64];
close(fd[1]);
read(fd[0], buf, strlen(messages[i]));
printf("%s\n", buf);
}
return 0;
}

	pipe	parent/потомок	сигнал/сигнал
соединение	+	-	+
сигнал/сигнал	сигнал/сигнал	сигнал/сигнал	сигнал/сигнал
копирование	-	-	+
сигнал/сигнал	сигнал/сигнал	сигнал/сигнал	сигнал/сигнал

сигнал/сигнал

3. Лаба UNIX: создание процесса в ОС UNIX и запуск новой программы (сх: именные вызовы);
Примеры программ из лабораторных работ, демонстрирующие эти действия. Процессы - «сироты», «зомби» и «демоны». Системные вызовы fork(), wait(), pipe(), signal() и sigaction();
назначение, примеры из лабораторных работ. Сравнение средств IPC: программных каналов, очередей сообщений и разделяемых сегментов.

Основная задача демонов - выполнение фоновых служебных задач и контроль других процессов ОС.
Правила программирования демона
Сброс маски режима создания файлов с помощью функции umask(). Демон, будучи наследником некоторого процесса, наследует от него маску режима создания файлов. Демон должен иметь возможность создавать файлы с любыми правами, потому маску предка следует сбросить. Вызвать fork() и завершить родительский процесс.
Создать новый сеанс с помощью функции setsid(). При этом процесс становится (а) лидером сеанса, (б) лидером группы, (в) лишается управляющего терминала.
Сделать корневой каталог текущим chdir(). Каталог, унаследованный от предка может находиться на смонтированной ФС. Если не сменить каталог, то ФС невозможно будет демонтировать до момента перезапуска системы. Закрывать файловые дескрипторы, унаследованные от родительского процесса. Открыть файловые дескрипторы 0, 1, 2 на /dev/null
Реализацию функции daemonize() смотри в исходном коде.
Единственность демона в системе
Демон должен существовать в системе в единственном экземпляре. В противном случае один и те же действия, иницируемые копиями одного и того же демона были дублировались, что могло бы привести к ошибке. Для гарантии единственности используется подход файл блокировки. already_running() используется для исключения повторного запуска демона с помощью файла блокировки. Каждая копия демона пытается создать файл и записать в него свой идентификатор. Если файл уже создан и заблокирован, то функция lockfile() вернет ошибку EACCESS или EAGAIN. В противном случае файл будет усечен до 0 и в него будет записан идентификатор. Усечение необходимо для стирания идентификатора предыдущего демона, который, возможно, был длиннее текущего.
ps -aux - вывод всех процессов и системных демонов
Стопбек Расшифровка
PPID - Идентификатор процесса родителя
PID - Идентификатор процесса
PGID - Идентификатор группы
SGID - Идентификатор сессии
TTY - Тип терминала
TPGID - Идентификатор группы терминалов
STAT - Состояние процесса
UID - Идентификатор пользователя
COMMAND - Команда, которой был запущен терминал
Если процесс находится в состоянии блокировки, это означает что он не получает процессорного времени.
Почему PPID имеет значение 1 (мб другой процесс, но не сильно большой) демона запускаем сами из терминала, его усыновляет процесс с номером 1 (или его потомок, если в итоге PPID не равен 1)
Почему PID == PGID == SID демон является лидером группы и лидером сессии
Почему TTY == ? нет управляющего терминала
Почему TPGID == -1 отсутствует терминальная группа (т.к. нет управляющего терминала)
Что за значения в STAT
"S" - ожидание завершения события (interruptable sleep - можно прервать сигналом, если вместо "S" - "D", то сон непрерываемый, ожидание завершения процесса IO).
"s" - лидер сессии
"T" - многопоточность

fd0, fd1, fd2 - что за файлы stdin, stdout, stderr
sigaction() - используется для изменения выполняемого процессом действия при получении определённого сигнала
SIGHUP - это сигнал, отправляемый процессу, когда его управляющий терминал закрыт.
sigemptyset() - инициализирует набор сигналов, указанный в set, и "очищает" его от всех сигналов.
С точки зрения ядра - терминальная сессия - это группа процессов, имеющих один идентификатор сеанса sid.
Группа процессов - инструмент для доставки сигнала нескольким процессам, а также сброс арбитража при доступе к терминалу. Идентификатор группы pid равен pid создавшего её процесса - лидера группы.
Сеанс - средство для контроля путем послышки сигналов над несколькими группами процессов со стороны терминального драйвера. Идентификатор сеанса sid равен sid создавшего его процесса - лидера группы.
Сеанс. Одновременно с сеансом создается новая группа с pgid равным pid лидера сеанса. Поскольку переход группы из сеанса в сеанс невозможен, то созданный сеанс процесс не может быть лидером группы.

int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
signum указывает сигнал и может быть любым допустимым сигналом, кроме SIGKILL и SIGSTOP
Если act не равно NULL, новое действие для сигнала устанавливается из act. Если значение oldact не равно NULL, предыдущее действие сохраняется в oldact.
Структура sigaction выглядит так:
struct sigaction {
void (*sa_handler)(int);
void (*sa_sigaction)(int, siginfo_t *, void *);
sigset_t sa_mask;
int sa_flags;
void (*sa_restorer)(void);
};
sa_handler и sa_sigaction вместе не назначать.
Полное sa_restorer не предназначено для использования приложением.
sa_handler определяет действие, которое должно быть связано с signum, и должно быть одно из следующих:
SIG_DFL для действия по умолчанию.
SIG_IGN, чтобы игнорировать этот сигнал.
Указатель на функцию обработки сигнала. Эта функция получает номер сигнала в качестве единственного аргумента.
sa_mask указывает маску сигналов, которые должны быть заблокированы во время выполнения обработчика сигнала. Кроме того, сигнал, который запустил обработчик, будет заблокирован, если только не используется флаг SA_NODEFER.
sa_flags определяет набор флагов, которые изменяют поведение сигнала.

int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
signum указывает сигнал и может быть любым допустимым сигналом, кроме SIGKILL и SIGSTOP
Если act не равно NULL, новое действие для сигнала устанавливается из act. Если значение oldact не равно NULL, предыдущее действие сохраняется в oldact.
Структура sigaction выглядит так:
struct sigaction {
void (*sa_handler)(int);
void (*sa_sigaction)(int, siginfo_t *, void *);
sigset_t sa_mask;
int sa_flags;
void (*sa_restorer)(void);
};
sa_handler и sa_sigaction вместе не назначать.
Полное sa_restorer не предназначено для использования приложением.
sa_handler определяет действие, которое должно быть связано с signum, и должно быть одно из следующих:
SIG_DFL для действия по умолчанию.
SIG_IGN, чтобы игнорировать этот сигнал.
Указатель на функцию обработки сигнала. Эта функция получает номер сигнала в качестве единственного аргумента.
sa_mask указывает маску сигналов, которые должны быть заблокированы во время выполнения обработчика сигнала. Кроме того, сигнал, который запустил обработчик, будет заблокирован, если только не используется флаг SA_NODEFER.
sa_flags определяет набор флагов, которые изменяют поведение сигнала.

3.2

```
#include <pthread.h>
#include <fcntl.h>

#define CONFIG_FILE "/etc/daemon.conf"
#define LOCKFILE "/private/var/run/daemon.pid"
#define LOCKMODE (S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH)

sigset_t mask;

// дописать смысл функций: fcntl, ftruncate,

void
daemonize(const char *cmd)
{
    int i, fd0, fd1, fd2;
    pid_t pid;
    struct rlimit rl;
    struct sigaction sa;

    // Сброс маски прав доступа
    umask(0);

    // Получение максимального числа файловых дескрипторов
    getrlimit(RLIMIT_NOFILE, &rl);

    // создание нового процесса и завершение родительского
    pid = fork()

    // Создание новой сессии (отвязка от терминала)
    setsid();

    // Обеспечить невозможность обретения управляющего терминала в будущем
    sa.sa_handler = SIG_IGN;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    if (sigaction(SIGHUP, &sa, NULL) < 0) {
        err_quit("%s: невозможно игнорировать сигнал SIGHUP", cmd);
    }

    chdir("/")

    if (rl.rlim_max == RLIM_INFINITY) {
        rl.rlim_max = 1024;
    }

    //закрытие всех открытых файловых дескрипторов
    for (i = 0; i < rl.rlim_max; i++) {
        close(i);
    }

    // перенаправление стандартных потоков в /dev/null
    fd0 = open("/dev/null", O_RDWR);
    fd1 = dup(0);
    fd2 = dup(0);

}

int
lockfile(int fd)
{
    struct flock fl;
    fl.l_type = F_WRLCK;
    fl.l_start = 0;
    fl.l_whence = SEEK_SET;
    fl.l_len = 0;
    return(fcntl(fd, F_SETLK, &fl));
}

int
already_running(void)
{
    int fd;
    char buf[16];

    fd = open(LOCKFILE, O_RDWR|O_CREAT, LOCKMODE);
    if (fd < 0) {
        syslog(LOG_ERR, "невозможно открыть %s: %s", LOCKFILE, strerror(errno));
        exit(1);
    }

    if (lockfile(fd) < 0) {
        if (errno == EACCES || errno == EAGAIN) {
            close(fd);
            return 1;
        }
        syslog(LOG_ERR, "невозможно установить блокировку на %s: %s",
            LOCKFILE, strerror(errno));
        exit(1);
    }

    ftruncate(fd, 0); // очищение содержимого файла
    sprintf(buf, "%ld", (long)getpid());
    write(fd, buf, strlen(buf)+1);
    return 0;
}

void
reread(void)
{
    // чтение конфигурационного файла
}

void *
thr_fn(void *arg)
{
    int err, signo;
    for (;;) {
        err = sigwait(&mask, &signo);
        if (err != 0) {
            syslog(LOG_ERR, "Ошибка вызова функции sigwait");
            exit(1);
        }
        switch (signo) {
            case SIGHUP:
                syslog(LOG_INFO, "Чтение конфигурационного файла");
                reread();
                break;
            case SIGTERM:
                syslog(LOG_INFO, "получен сигнал SIGTERM; выход");
                exit(0);
            default:
                syslog(LOG_INFO, "получен сигнал %d\n", signo);
        }
    }
    return 0;
}

int
main(int argc, char *argv[])
{
    int err;
    pthread_t tid;
    char *cmd;
    struct sigaction sa;

    if ((cmd = strrchr(argv[0], '/')) == NULL)
        cmd = argv[0];
    else
        cmd++;

    daemonize(cmd);

    if (already_running()) {
        exit(1);
    }

    sa.sa_handler = SIG_DFL;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGHUP, &sa, NULL)

    sigfillset(&mask);
    pthread_sigmask(SIG_BLOCK, &mask, NULL);
    pthread_create(&tid, NULL, thr_fn, 0);
}
```

II

2.2

II

1.2 ① fork

```
int main(void)
{
    pid_t childpid[2];
    for (int i = 0; i < 2; i++)
    {
        if ((childpid[i] = fork()) == -1)
        {
            perror("Can't fork.");
            exit(1);
        }
        else if (childpid[i] == 0)
        {
            printf("Child before sleep");
            sleep(2);
            printf("Child after sleep");
            return 0;
        }
        else
        {
            printf("Parent process");
        }
    }
    return 0;
}
```

② wait

```
for (int i = 0; i < 2; i++)
{
    if ((childpid[i] = waitpid(childpid[i], &status, 0)) == -1)
    {
        perror("Can't waitpid.n");
        exit(1);
    }
    printf("Child process has finished. Status = %d.\n", status);

    if (WIFEXITED(status))
    {
        printf("Child process has finished with code = %d",
            WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status))
    {
        printf("Child process has finished from signal with code = %d",
            WTERMSIG(status));
    }
    else if (WIFSTOPPED(status))
    {
        printf("Child process has stopped with code = %d.\n",
            WSTOPSIG(status));
    }
}
```

③ exec

```
б main:
char programs[2][32] = {"program_1.out", "program_2.out"};

б какому учена:
else if (childpid[i] == 0)
{
    exec(programs[i], "")
    printf("Child process");
    return 0;
}
```

④ pipe

```
int main(void)
{
    int status = 0;
    char msgs[2][128] = {"xxx", "yyy"};
    int fd[2];
    pid_t childpid[2];
    pipe(fd);
    for (int i = 0; i < 2; i++)
    {
        childpid[i] = fork()
        else if (childpid[i] == 0)
        {
            close(fd[0]);
            write(fd[1], msgs[i], strlen(msgs[i]));
            printf("Child process");
            return 0;
        }
        else
        {
            printf("Parent process");
        }
    }

    for (int i = 0; i < 2; i++)
    {
        (childpid[i] = waitpid(childpid[i], &status, 0)
        printf("Child process has finished. Status = %d.\n", status);

        if (WIFEXITED(status))
        {
            printf("Child process has finished with code = %d", WEXITSTATUS(status));
        }
        else if (WIFSIGNALED(status))
        {
            printf("Child process has finished from signal with code = %d", WTERMSIG(status));
        }
        else if (WIFSTOPPED(status))
        {
            printf("Child process has stopped with code = %d.\n", WSTOPSIG(status));
        }
    }

    close(fd[1]);
    read(fd[0], msgs[0], 128);
    printf("String from pipe: %s\n", msgs[0]);
    return 0;
}
```

Сигналы (processes), с помощью которых можно повлиять на выполнение программы) Сигнал - способ информирования процесса ядром о происшествии какого-то события. Если возникает несколько одностипных событий, процесс будет ловить только один сигнал. Сигнал означает, что произошло событие, но ядро не сообщает ничего таких событий произошло. Обычно, получение процессом сигнала предписывает ему завершиться. Вместе с тем процесс может установить собственную реакцию на полученный сигнал. Например, получив сигнал процесс может его проигнорировать, или вызвать на выполнение некоторую программу, а после ее завершения продолжить выполнение с точки получения сигнала. Установить реакцию процесса на поступающий сигнал можно с помощью системного вызова signal(): func = signal(sign, function); Где sign - номер сигнала, а function - адрес функции, которая должна быть выполнена при поступлении указанного сигнала. Возвращаемое значение - адрес функции, которая будет реагировать на поступление сигнала. Возможны три типа реакции процесса на сигнал:

1. Игнорирование			
2. Действие по умолчанию			
3. Определение пользовательского решения			
SIGHUP 1	Разрыв связи с терминалом	SIGTERM 15	kill() в режиме терминала
SIGINT 2	Ctrl+C	SIGUSR1 16	пользовательские
SIGQUIT 3	Ctrl+^	SIGUSR2 17	Тоже пользовательские
SIGKILL 9	Вызов системного kill()	SIGDCL 18	При завершении потока
SIGSEGV 11	Сопровождает выход процесса за пределы сегмента	SIGPWR 19	Падение напряжения
SIGSYS 12	Ошибка выполнения системного вызова	SIGDFL 0	0
SIGPIPE 13	Нет читающего процесса	SIGIGN 1	1
SIGALARM 14	Истекло время, установленное alarm()		

signal (номер) - 2 бунет; sigaction - 3 бунет.

В UNIX 2 средства послыки и приема сигналов: системные вызовы kill и signal. int kill(int pid, int sig); pid = 1 => sig посылается группе процессов pid = 0 => sig посылается всем процессам группы вызв. kill, кроме проц-ов с pid=0 и pid=1 pid = -1 => sig посылается sig процессу с тем же pid, что и вызв. kill. void (*signal(int sig, void(*handler)(int)))(int); В рез-те получения сигнала вызывается handler. signal не входит в POSIX, но входит в ANSI C => есть в любой поставке, но реакция на сигналы может отличаться в разных ОС. signal возвращает указатель на старый обработчик.

```
#include <signal.h>
int main()
{
    void (*old_handler)(int) = signal(SIGINT, SIGIGN);
    // обработка
    signal(SIGINT, old_handler);
}

int sigaction(int sig_num, struct sigaction *action, struct sigaction *old_action);
Вх. в POSIX
В отличие от signal работает со структурой.

#include <signal.h>
int main(void)
{
    struct sigaction old_action, new_action;
    sigaction(SIGINT, NULL, &old_action);
    new_action.sa_handler = new_handler;
    sigaction(SIGINT, &new_action, NULL);
}
```

4 Лаба «Производство-потребление». Задача «Обедающие философы» — модели распределения ресурсов вычислительной системы. Поддержка семафоров в системе UNIX/Linux. Задача «Производство-потребление», множественные семафоры и алгоритм Эд. Дейкстры «Производство-потребление», реализация на семафорах в системе UNIX/Linux.

Обедающие философы

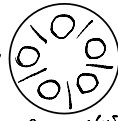
На круглом столе стоит 5 тарелок. Между ними лежит по 1 прибору.

Каждый философ кушает и мыслит.

Три алгоритма:

- Каждый из философов пытается взять сразу 2 прибора, если складет приборы!
- Каждый берет сначала левый. Если не удалось взять правый, кладет левый на место.
- Каждый берет сначала левый. Если не удалось взять правый, кладет левый на место.

- 1) бес. отладки выжили
- 2) dead lock
- 3) захват освобождение одним и тем же ресурсом



семафоры есть в теории.

```
var forks : array [1..5] of semaphore;
i : integer;
begin
  i := 5;
  repeat
    forks[i] := 1;
    i := i-1;
  until i = 0;
  cobegin
    1: begin
      left := 1;
      right := 5;
    end;
    5: begin
      left := 5;
      right := 1;
    end;
  and
    repeat
      // размышляют
      P(forks[left], forks[right]);
      // ест
      V(forks[left], forks[right]);
    forever
  end;
end
```

Задача производства-потребления.

Два типа процессов: Производители производят продукт и помещают в буфер, Потребители забирают продукт из буфера.

Семафоры Дейкстры — неогранич. запасы переменных.

Р(1) V(1) P(1) S=S-1, S>0, иначе захватить S=S+1 V(1) S=S+1

Если S=0, P — думать, если S=неотр. вел. — считать. P и V — неделимы. У семафора нет хозяина, его может освободить любой процесс, знающий о нем.

P1: P(1) V(1) CR1 V(1) ... P2: P(1) V(1) CR2 V(1) ...

bfull, bempty: integer; // считающие
binary: integer; // бинарные

prod: ... P(bempty) // захв. своб. места P(binary) // положить V(binary) // увел. число заполн. ячеек V(bfull) // гудет-блок cons: ... P(bfull) // гудет-блок P(binary) // положить V(binary) V(bempty)

5 Лаба «Писатели-читатели», UNIX — монитор Хоара, решение с использованием семафоров Unix и разделяемой памяти, пример реализации из лабораторной работы.

```
#include <sys/shm.h>
#include <sys/sem.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <sys/signal.h>
```

```
#define SHM_KEY 123
```

```
#define WR_CNT 3
```

```
#define RD_CNT 3
```

```
#define P -1
```

```
#define V 1
```

```
int flag = 1;
```

```
struct sembuf start_read[] =
{
  {READ_QUEUE, V, 0},
  {BIN_WRITING, 0, 0},
  {WRITE_QUEUE, 0, 0},
  {ACTIVE_READERS, V, 0},
  {READ_QUEUE, P, 0}
};
```

```
struct sembuf stop_read[] =
{
  {ACTIVE_READERS, P, 0};
};
```

```
struct sembuf start_write[] =
{
  {WRITE_QUEUE, V, 0},
  {ACTIVE_READERS, 0, 0},
  {ACTIVE_WRITER, P, 0},
  {BIN_WRITING, V, 0},
  {WRITE_QUEUE, P, 0}
};
```

```
struct sembuf stop_write[] =
{
  {BIN_WRITING, P, 0},
  {ACTIVE_WRITER, V, 0};
};
```

```
void handle_sig(int signum)
{
  if (getppid() == getppgr())
  {
    flag = 0;
    printf("pid=%d caught sig=%d\n", getpid(), signum);
  }
}
```

```
void writer(char *shared_char, int semid)
{
  semop(semid, start_write, 5);

  ++(*shared_char);
  if (*shared_char > 'z')
    *shared_char -= 26;

  semop(semid, stop_write, 2);
}
```

```
void reader(char *shared_char, int semid)
{
  semop(semid, start_read, 5);
  printf("> reader=%d GET char=%c\n", getpid(), *shared_char);
  semop(semid, stop_read, 1);
}
```

```
int main()
{
  pid_t child1[WR_CNT + RD_CNT];
  pid_t wpid;
  int wstatus;
  int perms = S_IRUSR | S_IWUSR | S_IRGRP;
  char *shared_char;

  if (signal(SIGINT, handle_sig) == SIG_ERR)
  {
    perror("cannot signal");
    exit(1);
  }
}
```

```
int shmid = shmget(SHM_KEY, sizeof(char), IPC_CREAT | perms);
shared_char = (char *)shmat(shmid, 0, perms);
int semid = semget(SEM_KEY, SEM_CNT, IPC_CREAT | perms);

*shared_char = 'z';

semctl(semid, ACTIVE_READERS, SETVAL, 1)
semctl(semid, ACTIVE_WRITER, SETVAL, 1)
semctl(semid, READ_QUEUE, SETVAL, 0)
semctl(semid, WRITE_QUEUE, SETVAL, 0)
semctl(semid, BIN_WRITING, SETVAL, 0)
```

```
for (size_t i = 0; i < WR_CNT; ++i)
{
  child1[i] = fork();
  if (child1[i] == -1)
  {
    perror("cannot fork writer");
    exit(1);
  }
  else if (child1[i] == 0)
  {
    writer(shared_char, semid);
  }
}

for (size_t i = 0; i < RD_CNT; ++i)
{
  child1[i] = fork();
  if (child1[i] == -1)
  {
    perror("cannot fork reader");
    exit(1);
  }
  else if (child1[i] == 0)
  {
    reader(shared_char, semid);
  }
}

for (size_t i = 0; i < (RD_CNT + WR_CNT); ++i)
{
  pid_t wpid = waitpid(child1[i], &wstatus, MUNTRACED);
  if (WIFEXITED(wstatus))
    printf("pid=%d exited with status=%d\n", wpid, WEXITSTATUS(wstatus));
  else if (WIFSIGNALED(wstatus))
    printf("pid=%d killed by sig=%d\n", wpid, WTERMSIG(wstatus));
  else if (WIFSTOPPED(wstatus))
    printf("pid=%d stopped by sig=%d\n", wpid, WSTOPSIG(wstatus));
}

semctl(semid, 1, IPC_RMID, NULL)
shmdt(shared_char)
shmctl(semid, IPC_RMID, NULL)

return 0;
}
```

Мониторинг — программ. средство для над. Хоарами. Идея: создать мех. который идентифицирует (дискриминирует) потоки паралл. процессов, идущих в циклоп. отсутствие критич. данных.

Монитор — язык конструкции, состоящий из структуры данных и набор. правил, которые управ. данными. Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу.

Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу.

Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу.

Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу. Монитор — механизм, который управ. доступом к ресурсу.

6 Лаба «Писатели-читатели» на базе Win32 API. Типы событий и семафоров (семафоры «против» мьютексов).

Монитор читателя-писателя

Процессы-читатели могут читать параллельно, не имея друг друга. Процесс-писатель имеет монополиз. доступ к данным. При этом любой ресурс можно вернуть в pool свободных ресурсов.

Монитор — это набор разделяемых переменных и повторно входимых процедур доступа к ним, которыми процессы пользуются в режиме разделения, причем в каждый момент времени им может пользоваться только один процесс.

это механизм организации параллелизма, который содержит как данные, так и процедуры, необходимые для динамического распределения конкретного общего ресурса или группы общих ресурсов.

События в Windows: запрос ресурса — происходит возобновление работы независимых процессов. запрос ресурса — происходит возобновление работы независимых процессов.

HANDLE hEvent = CreateEvent(NULL, FALSE, FALSE, NULL);

SetEvent(hEvent); WaitForSingleObject(hEvent, INFINITE);

HANDLE hEvent = CreateEvent(NULL, TRUE, FALSE, NULL);

SetEvent(hEvent); WaitForSingleObject(hEvent, INFINITE);

ResetEvent(hEvent);

```
#include "Hoare.h"
#include <stdio.h>
#include <stdint.h>
#include <time.h>
#include <stdlib.h>
```

```
#define READER_COUNT 5
#define WRITER_COUNT 4
#define WORK_ITER 10
```

```
HANDLE mayReadEvent;
HANDLE mayWriteEvent;
HANDLE mutex;
```

```
int number_of_readers = 0;
int writer_queue = 0;
int reader_queue = 0;
int running = 1;
char data = 'a';
```

```
void start_read()
{
  InterlockedIncrement(&reader_queue);
  if (writer_queue > 0)
  {
    WaitForSingleObject(mayReadEvent, INFINITE);
  }
  InterlockedDecrement(&reader_queue);
  InterlockedIncrement(&number_of_readers);
}
```

```
void stop_read()
{
  InterlockedDecrement(&number_of_readers);
  if (number_of_readers == 0)
    SetEvent(mayWriteEvent);
}
```

```
void start_write()
{
  WaitForSingleObject(mutex, INFINITE);
  InterlockedIncrement(&writer_queue);
  if (number_of_readers > 0)
  {
    WaitForSingleObject(mayWriteEvent, INFINITE);
  }
  InterlockedDecrement(&writer_queue);
  ReleaseMutex(mutex);
}
```

```
void stop_write()
{
  ResetEvent(mayWriteEvent);
  if (reader_queue > 0)
    SetEvent(mayReadEvent);
  else
    SetEvent(mayWriteEvent);
}
```

```
DWORD WINAPI ReaderLoop(PVOID pvParam)
{
  DWORD proc = GetCurrentThreadId();
  start_read();
  stop_read();
}
```

```
DWORD WINAPI WriterLoop(PVOID pvParam)
{
  DWORD proc = GetCurrentThreadId();
  start_write();
  data++;
  if (data > 'z') data = 'a';
  stop_write();
}
```

```
int main()
{
  mayReadEvent = CreateEvent(NULL, FALSE, FALSE, NULL);
  mayWriteEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
  mutex = CreateMutex(NULL, FALSE, NULL);
}
```

```
HANDLE hThread[READER_COUNT + WRITER_COUNT];
DWORD threadIDs[READER_COUNT + WRITER_COUNT];
for (int i = 0; i < READER_COUNT; i++)
{
  hThread[i] = _beginthreadex(NULL, 0, ReaderLoop,
    NULL, 0, threadIDs + i);
}
```

```
for (int i = READER_COUNT; i < READER_COUNT + WRITER_COUNT; i++)
{
  hThread[i] = _beginthreadex(NULL, 0, WriterLoop,
    NULL, 0, threadIDs + i);
}
```

```
WaitForMultipleObjects(READER_COUNT + WRITER_COUNT,
  hThread,
  FALSE,
  INFINITE);
```

```
for (int i = 0; i < READER_COUNT + WRITER_COUNT; i++)
{
  CloseHandle(hThread[i]);
}
```

```
CloseHandle(mayReadEvent);
CloseHandle(mayWriteEvent);
CloseHandle(mutex);
return 0;
}
```

ключевые различия между мьютексом и семафором:

Владение: Мьютекс имеет строгое понятие владения: поток, который его захватил, должен сам его освободить. Семафор не имеет понятия владения: один поток может выполнить P(), а другой V()

Счетчик: Мьютекс бинарный — может быть только в состоянии 0 или 1 (заблокирован/разблокирован). Семафор может иметь произвольное целочисленное значение счетчика (>0)

Назначение: Мьютекс используется для взаимного исключения (mutual exclusion) — защиты разделяемого ресурса. Семафор для более общей синхронизации процессов и потоков (сигнализирование о событиях)

Поведение: Мьютекс должен быть освобожден тем же потоком, который его захватил. Семафор можно уменьшать и увеличивать из разных потоков/процессов

Читатель-писатель. Онок не без бага, а уноз шесто

```
RESOURCE MONITOR;
var
  active_readers : integer;
  active_writer : logical;
  can_read, can_write : conditional;
procedure star_read
begin
  if (active_writer or turn(can_write)) then
    wait(can_read);
  active_readers++;
  signal(can_read);
end;
procedure stop_read
begin
  active_readers--;
  if (active_readers == 0) then signal(can_write);
end;
procedure start_write
begin
  if ((active_readers > 0) or active_writer) then wait(can_write);
  active_writer := true;
end;
procedure stop_write
begin
  active_writer = false;
  if (readers.queue) then
    if (readers.queue) then
      signal(can_read);
    else
      signal(can_write);
    end;
  begin
    active_readers = 0;
    active_writer = false;
  end;
```

• Читатель-смотить. Моит
итать / и не мешают
другим / не мешают
другим / не мешают
другим / не мешают

• Писатель-изменить
имеет монитон
актуп к
критиче

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <unistd.h>

#define SIZE_BUF 26

/* семафоры */
#define BIN_SEM 0
#define BUFF_FULL 1
#define BUFF_EMPTY 2

/* операции над семафорами */
#define P -1
#define V 1

struct sembuf start_produce[2] = {{BUFF_EMPTY, P, 0}, {BIN_SEM, P, 0}};
struct sembuf stop_produce[2] = {{BIN_SEM, V, 0}, {BUFF_FULL, V, 0}};
struct sembuf start_consume[2] = {{BUFF_FULL, P, 0}, {BIN_SEM, P, 0}};
struct sembuf stop_consume[2] = {{BIN_SEM, V, 0}, {BUFF_EMPTY, V, 0}};

int sig_flag = 1;

void producer(char **producer_ptr, char *alpha_ptr, const int semid) {
  semop(semid, start_produce, 2)

  /* записываем букву в буфер */
  **producer_ptr = *alpha_ptr;

  if (*alpha_ptr == '\x')
    *alpha_ptr = 'a';
  else
    ++(*alpha_ptr);

  ++(*producer_ptr);

  semop(semid, stop_produce, 2)
}

void consumer(char **consumer_ptr, const int semid) {
  semop(semid, start_consume, 2)

  ++(*consumer_ptr);

  semop(semid, stop_consume, 2)
}

int main(void) {
  key_t shmkey = ftok("file.txt", 3);
  int perms = S_IRWXU | S_IRWXG | S_IRWXXO;

  /* выделение разделяемой памяти */
  int shmid = shmget(shmkey, (SIZE_BUF + 3) * sizeof(char), perms | IPC_CREAT);

  /* подключение сегмента */
  char *addr = shmat(shmid, NULL, 0);

  char **producer_ptr = (char **) addr;
  char **consumer_ptr = producer_ptr + sizeof(char);
  char *alpha_ptr = (char*) (consumer_ptr + sizeof(char));

  *consumer_ptr = alpha_ptr + sizeof(char);
  *producer_ptr = *consumer_ptr;
  *alpha_ptr = 'a';

  key_t semkey = ftok("file.txt", 4);

  int semid = semget(semkey, 3, IPC_CREAT | perms);

  semctl(semid, BIN_SEM, SETVAL, 1)
  semctl(semid, BUFF_EMPTY, SETVAL, SIZE_BUF)
  semctl(semid, BUFF_FULL, SETVAL, 0)

  /* PID процесса */
  pid_t cpid[PROD_CNT + CONS_CNT];

  for (int i = 0; i < PROD_CNT; i++) {
    cpid[i] = fork();
    if (cpid[i] == 0)
      producer(producer_ptr, alpha_ptr, semid);
  }

  for (int j = 0; j < CONS_CNT; j++) {
    int i = j + PROD_CNT;
    cpid[i] = fork();
    if (cpid[i] == 0)
      consumer(consumer_ptr, semid);
  }

  for (int i = 0; i < (PROD_CNT + CONS_CNT); i++) {
    int wait_status;
    pid_t res_waitpid = waitpid(cpid[i], &wait_status, WUNTRACED | WCONTINUED);
  }

  shmdt(addr)
  semctl(semid, 1, IPC_RMID, NULL)
  shmdt(shmid, IPC_RMID, NULL)

  return 0;
}
```

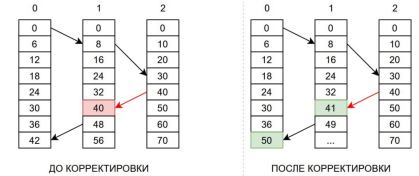
Алгоритм Token Ring
Процессы объединены в кольцо и передают по нему токен друг другу. Токен — сообщение, которое соответствует некоторому разделяемому ресурсу. Если текущему процессу ресурс не нужен, то он передает токен далее по кольцу, если нужен — удерживает его до тех пор, пока использует соответствующий ресурс, затем передает далее.

Транзакции
Транзакция — последовательность действий над объектами, переводящая систему между целостными состояниями. Свойства транзакции (ACID):
Атомарность
Транзакция фиксируется (commit) и может быть отменена в начальное состояние в случае хотя бы одной ошибки (rollback).
Согласованность
Пример: Если товар куплен, то это должно быть отражено в системе складского учета
Изоляция
Прочие процессы не имеют доступа к транзакции
Устойчивость
В ходе транзакции ни один сбой не может повредить данные, участвующие в транзакции.

Механизмы транзакций
Рабочее пространство
Для процесса, участвующего в транзакции, создается рабочее пространство, в которое копируются все данные, необходимые для совершения транзакции. Данный способ требует больших накладных расходов, обусловленных копированием данных.
Список намерений.
Данный механизм предусматривает наличие в системе журнала. Во время транзакции данные изменяются непосредственно, без копирования. При этом в журнале фиксируется транзакция, начальное и конечное значение измененного ей данного. В случае отсутствия записи о фиксации транзакции, происходит ее откат по журналу.



Особенности взаимодействия процессов в распределенных системах
Задача синхронизации процессов в распределенных системах заключается в том, чтобы «договориться» о времени, то есть установить отношение «случилось до —> случилось после». Один из алгоритмов, использующийся для синхронизации при обмене сообщениями — это алгоритм временных отметок Лампорта.
Идея алгоритма: В отправленное сообщение добавляется локальное время отправки T_send. Хост, получающий сообщение, сравнивает T_send и локальное время получения сообщения T_rec. Если T_rec <= T_send, то T_rec = T_send + 1 и дальнейший отсчет времени на хосте, получающем сообщение, ведется от нового (инкрементирования) T_rec.



Алгоритмы взаимного исключения в распределенных системах
Централизованный алгоритм (алгоритм Забияки)
Идея: Наличие в локальной распределенной сети процесса-координатора P_coord. Его задача в предоставлении монопольного доступа к разделяемым ресурсам. Некоторый процесс P обращается к P_coord с запросом на разделяемый ресурс, передавая его идентификатор. Если ресурс свободен, то P получает доступ, иначе блокируется. В случае запроса одного и того же ресурса одновременно несколькими процессами, доступ получит тот, кто пришел раньше.
(!) Возможно аварийное завершение P_coord. Учитывая это обстоятельство, процессы отправляют запрос на получение доступа с таймаутом. Если он истек и ответ не получен — происходит переизбор P_coord. P посылает процессам системы запрос на становление новым координатором и свой номер. Процессы, получившие такое сообщение, сравнивают пришедший номер с собственным. Если их собственный номер больше — они рассылают аналогичное сообщение. В результате такого голосования новым координатором становится процесс, обладающий наибольшим номером.
Распределенный алгоритм
Пусть процесс P взаимодействует с N процессами. P рассылает всем N процессам запрос на доступ к некоторому разделяемому ресурсу. Пусть некоторый P_other получил такое сообщение. Если P_other не использует и не собирается использовать ресурс, то он отправляет ответ ОК. Если P_other использует ресурс, то сообщение не отправляется. Если P и P_other делают запрос на ресурс одновременно, то раньше доступ получит тот, кто сделал запрос раньше. Если P получил N ответов ОК, то он захватывает ресурс, иначе он помещается в очередь процессов, ожидающих ресурса.

9.2

II

8.2

II

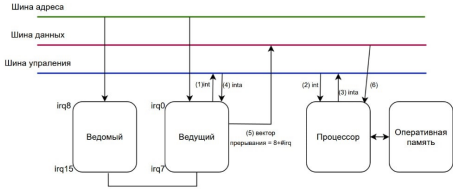
7.2

II

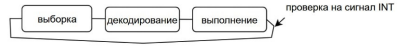
В теор.

DOS INT8h. Трех шинная концептуальная архитектура. Программируемый контроллер прерываний. Адресация аппаратных прерываний в простейшей схеме (16-разрядный реальный режим). Функции обработки прерывания от системного таймера в ОС разделения времени.

Обработчик прерывания DOS INT8h. Описание и функции. ЭВМ имеет системный таймер. Это физическое устройство подключено к линии запроса на прерывание IRQ0 (смотри концептуальную трех шинную архитектуру) и инициирует выполнение его обработчика 18,2 раза в секунду. Это аппаратное прерывание, так как оно инициировано аппаратурой – системным таймером. При запуске системы устанавливается свой обработчик данного прерывания, который выполняет три функции: Инкремент счетчика тиков таймера. Это двойное слово (4 байта) по адресу 0000:046Ch. Если с момента запуска таймера прошло более 24-х часов, происходит переполнение и в ячейку по адресу 0000:0470h заносится значение 1. Контроль моторчика дисководов. Если обращения к моторчику дисководов не было более 2-х секунд, обработчик инициирует его отключение. Ячейка 0000:0440h содержит время до его отключения. Когда время становится равным нулю, происходит отключение. Вызов прерывания INT 1Ch. При инициализации обработчик INT 1Ch содержит IRET, то есть ничего не делает. Данный обработчик может быть модифицирован для выполнения периодических действий. Во время выполнения INT 1Ch аппаратные прерывания запрещены. Контроллер прерываний В шинной архитектуре (еще рассматривали канальную) для управления внешними устройствами используются контроллеры Контроллер – это программно-управляемое физическое устройство. Процессор инициирует операцию ввода-вывода путем отправки на контроллер соответствующей команды, после чего контроллеру передается управление вводом-выводом и процессор не прерывается. После окончания операции ввода-вывода контроллер прерывания PIC выставляет на шину управления сигнал INT, процессор принимает этот сигнал и выставляет на шину управления сигнал INTA. Контроллер прерывания принимает данный сигнал и выставляет на шину данных вектор прерывания – номер прерывания в таблице векторов прерываний, который вычисляется как БАЗА + номер линии IRQ. База от ведущей части PIC равна 8, а номер линии – цифра после IRQ. В случае прерывания системного таймера его линия IRQ0, тогда вектор прерывания равен 0 + 8 = 8. Вектор прерывания – смещение его адреса в таблице векторов прерываний системы.



Процессор проверяет наличие сигнала INT после выполнения каждой команды:



Маскируемые и немаскируемые прерывания Маскируемое прерывание может быть проигнорировано, если флаг IF регистра флагов сброшен. В обработчике INT8h запрет маскируемых прерываний происходит в двух местах: в самом начале обработчика и перед сбросом контроллера прерываний (Чтобы не допустить вызова обработчика INT8h). Префиксная команда lock делает выполнение следующей после нее команды неделимой путем блокировки локальной шины памяти. Lock стоит перед and, которая требует 2 обращения к памяти. Отложенное действие в обработчике Отложенным действием является отключение моторчика дисководов путем отправки команды в порт. Разгон диска – времязатратное действие, потому его остановка инициируется только в том случае, если операций над ним не было достаточно долго. Основные функции обработчика прерываний системного таймера в системах разделения времени

	UNIX	Windows
По тикку	– Счет тиков аппаратного таймера – Декремент кванта текущего потока. – Инкремент времени использования процессора – Нахождение за списком отложенных вызовов	– Инкремент счётчика системного времени – Декремент счётчиков отложенных задач – Декремент остатка кванта текущего потока.
По главному тикку	– Добавление в очередь на выполнение функций, относящихся к работе планировщика-диспетчера (например, пересчет приоритетов) – Пробуждение системных процессов, таких, как явгернт и рядбедомс (процедура wakeip перемещает дескрипторы процессов из очереди «спящих» в очередь «готовых к выполнению») – Обработка сигналов тревоги: декремент будильников и измерение времени работы процесса	– Инициализация диспетчера устройств баланс (освобождение объекта «события» каждую секунду)
По кванту	– Псылка текущему процессу сигнала SIGXCPU, если присвоенный выделенный ему квант процессорного времени.	– Инициализация диспетчеризации потоков (посредством добавления соответствующего объекта в очередь DPC)

bakery.x:

```
const ADD = 0;
const SUB = 1;
const MUL = 2;
const DIV = 3;

/* Универсальная структура для всех операций */
struct bakery {
    int client_number; /* Номер клиента в очереди */
    char operation; /* Операция */
    double arg1; /* Первый аргумент */
    double arg2; /* Второй аргумент */
};
```

```
program BAKERY_PROG {
    version BAKERY_VER {
        int GET_NUMBER(void) = 1; /* Возвращает номер в очереди */
        double CALCULATE(bakery) = 2; /* Выполняет вычисления */
    } = 1;
} = 0x20000001;
```

bakery - client.c

/*
 * This is sample code generated by rpcgen.
 * These are only templates and you can use them
 * as a guideline for developing your own functions.
 */

```
#include "bakery.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
```

```
double get_random_number() {
    return (rand() % 21) - 10;
}
```

```
char get_random_operation() {
    return (char)(rand() % 4);
}
```

```
void bakery_prog_1(char *host)
{
    CLIENT *clnt;
    int *number;
    bakery calc_req;
    double *calc_resp;

    srand(time(NULL));

    clnt = clnt_create(host, BAKERY_PROG, BAKERY_VER, "udp");
    if (clnt == NULL) {
        clnt_perror(clnt, "Ошибка получения номера");
        exit(1);
    }
```

```
    while (1) {
        sleep(1);

        number = get_number_1(NULL, clnt);
        if (number == NULL) {
            clnt_perror(clnt, "Ошибка получения номера");
            exit(1);
        }

        printf("Получен номер: %d\n", *number);
        sleep(2);
```

```
        /* Заполняем структуру bakery */
        calc_req.client_number = *number;
        calc_req.operation = get_random_operation();
        calc_req.arg1 = get_random_number();
        calc_req.arg2 = get_random_number();
```

```
        /* Выводим информацию об операции */
        printf("Операция: ");
        switch(calc_req.operation) {
            case ADD: printf("ADD"); break;
            case SUB: printf("SUB"); break;
            case MUL: printf("MUL"); break;
            case DIV: printf("DIV"); break;
        }

        printf("\nПервое число: %.2lf\n", calc_req.arg1);
        printf("Второе число: %.2lf\n", calc_req.arg2);
```

```
        // Если при делении второе число равно 0
        while (calc_req.operation == DIV && calc_req.arg2 == 0) {
            calc_req.arg2 += 1;
            printf("Второе число изменено на: %.2lf для предотвращения деления на ноль\n", calc_req.arg2);
        }
```

```
        calc_resp = calculate_1(&calc_req, clnt);
        if (calc_resp == NULL) {
            clnt_perror(clnt, "Ошибка коммуникации");
            exit(1);
        }
        if (*calc_resp == ERROR_RESULT) {
            printf("Ошибка вычисления на сервере\n");
        }
```

```
        printf("Результат: %.2lf ", calc_req.arg1);
        switch(calc_req.operation) {
            case ADD: printf("+"); break;
            case SUB: printf("-"); break;
            case MUL: printf("*"); break;
            case DIV: printf("/"); break;
        }

        printf(" %.2lf = %.2lf\n", calc_req.arg2, *calc_resp);
    }
```

```
    clnt_destroy(clnt);

    int main(int argc, char *argv[])
    {
        char *host;

        if (argc < 2) {
            printf("Использование: %s server_host\n", argv[0]);
            exit(1);
        }

        host = argv[1];
        bakery_prog_1(host);
        return 0;
    }
```

RPC являются средством взаимодействия параллельных процессов (IPC - Inter-process communication). Как известно, в самом общем случае различаются два вида взаимодействия параллельных процессов (IPC):

- 1. Вызов локальных процедур (LPC – local procedure call)
 - 2. Вызов удаленных процедур (RPC – remote procedure call)
- Вызов удаленных процедур сделан максимально подобным вызову локальных процедур. В ядре RPC не используют протокол, в чем и состоит принципиальное различие с сокетами. Действия фактически выполняются не по протоколу. Действия выполняются в сети путем связывания портов между собой.

12.2

II

bakery_server.c:

```
/*
 * This is sample code generated by rpcgen.
 * These are only templates and you can use them
 * as a guideline for developing your own functions.
 */

#include <stdbool.h>
#include <time.h>
#include <stdio.h>
#include "bakery.h"

#define MAX_CLIENTS 30
#define TIMEOUT 2

bool choosing[MAX_CLIENTS] = {0};
int number[MAX_CLIENTS] = {0};
int last_num = 0;
static int current_number = 0;

int *
get_number_1_svc(void *argp, struct svc_req *rqstp)
{
    static int result;
    setbuf(stdout, NULL);

    current_number++;
    result = current_number;

    int i = (current_number - 1) % MAX_CLIENTS;
    choosing[i] = true;
    number[i] = current_number;
    choosing[i] = false;

    printf("SERVER || Выдан номер: %d\n", result);
    return &result;
}

double *
calculate_1_svc(bakery *argp, struct svc_req *rqstp)
{
    static double result;
    setbuf(stdout, NULL);
    printf("SERVER || Обработка клиента с номером %d\n", argp->client_number);

    int i = (argp->client_number - 1) % MAX_CLIENTS;
    time_t start, end;

    for (int j = 0; j < MAX_CLIENTS; j++)
    {
        while (choosing[j]);

        if (last_num > number[i])
        {
            number[i] = 0;
            result = -1;
            return &result;
        }

        start = clock();
        while ((number[j] > 0) && (number[j] < number[i] ||
            (number[j] == number[i] && j < i)))
        {
            end = clock();
            if ((end - start) / CLOCKS_PER_SEC > TIMEOUT)
                break;
        }

        switch (argp->operation)
        {
            case ADD:
                result = argp->arg1 + argp->arg2;
                break;
            case SUB:
                result = argp->arg1 - argp->arg2;
                break;
            case MUL:
                result = argp->arg1 * argp->arg2;
                break;
            case DIV:
                result = argp->arg1 / argp->arg2;
                break;
            default:
                result = ERROR_RESULT;
                printf("Неизвестная операция\n");
        }

        last_num = number[i];
        number[i] = 0;
        printf("SERVER | Клиент %d обработан, результат: %lf\n",
            argp->client_number, result);

        for (int j = 0; j < MAX_CLIENTS; j++)
            if (number[j] > 0)
                return &result;

        last_num = 0;
        return &result;
    }
}
```

rpcgen -a calculator.x

Makefile.calculator: используется для компиляции кода клиента и кода сервера;
calculator.h: объявление используемых переменных и функций;
calculator_xdr.c: кодировка нестандартных типов данных;
calculator_clnt.c: прокси-сервер удаленного вызова
вызывается локальной операционной системой, параметры вызова упакованы в сообщение. Далее сообщение посылается на сервер.
Данный файл генерируется с помощью *rpcgen* и не нуждается в модификации;

calculator_svc.c: преобразует запрос, вводимый по сети, в вызов локальной процедуры. Файл ответственный за распаковку полученного сервером сообщения и вызов фактической реализации на стороне сервера и на уровне приложения. Не нуждается в модификации;

calculator_client.c: скелетон программы клиента, требуется модифицировать;
calculator_server.c: скелетон программы сервера, требуется модифицировать;

11.2

II

10.2

II